

*Same as well
Just better*

SAME YOU SINCE 2009
JUST BETTER

Before we start

Let's talk about your Projects

Lesson 7

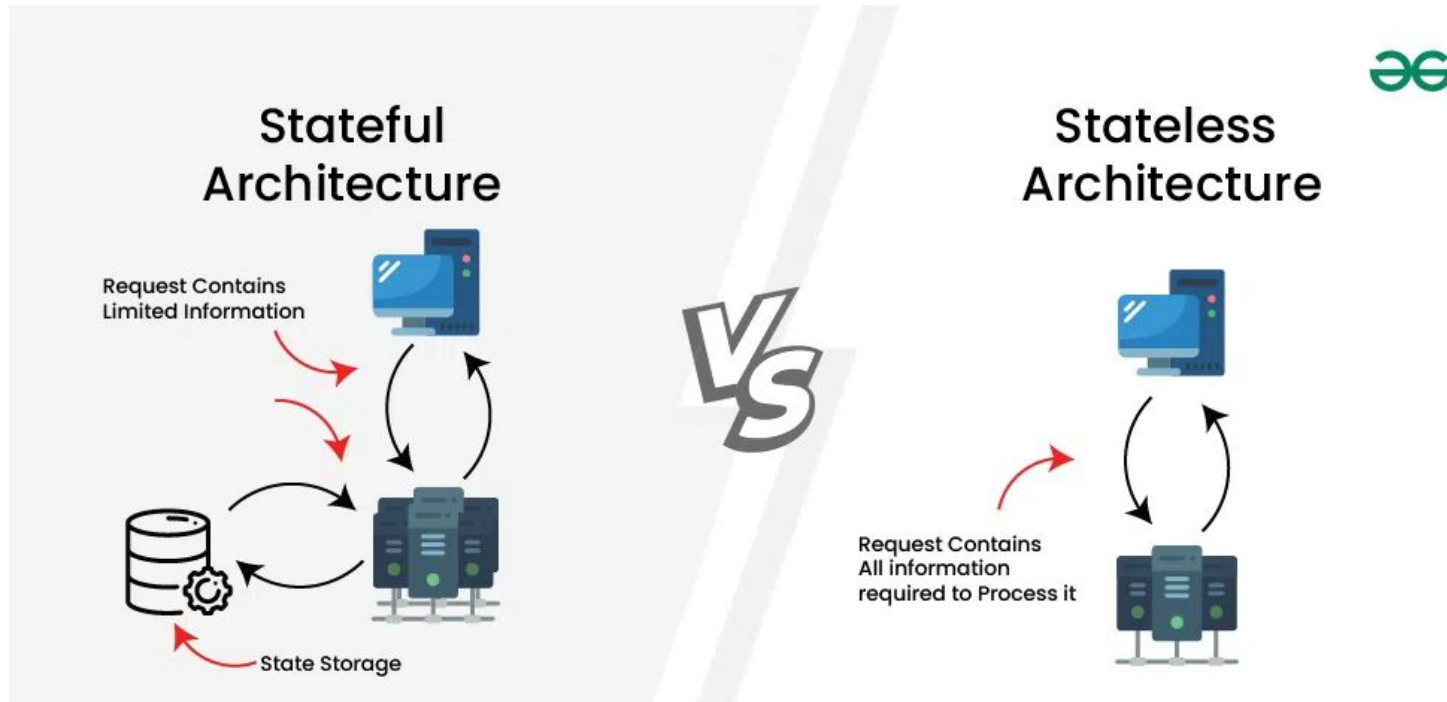
Python for DevOps Part 3 DB

Your Project

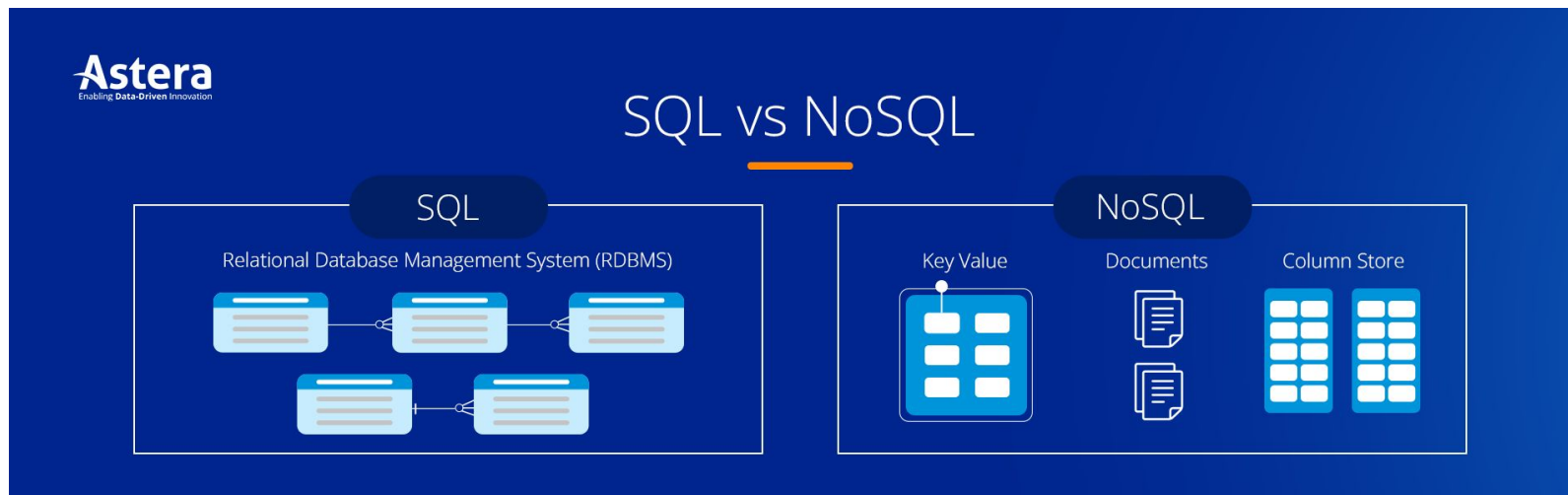
- You have a running web framework (FastAPI/Flask).
- You have a /health endpoint for future K8s readiness checks.
- You are fetching external data using requests.
- Problem: When the server restarts, all data disappears!



- Stateless Apps: Don't save internal data between requests. Good for scaling, bad for remembering users.
- Stateful Apps: Rely on saved data (State).
- To build real products, we need to save user inputs, cached API responses, and configurations.
- The solution: A Database.

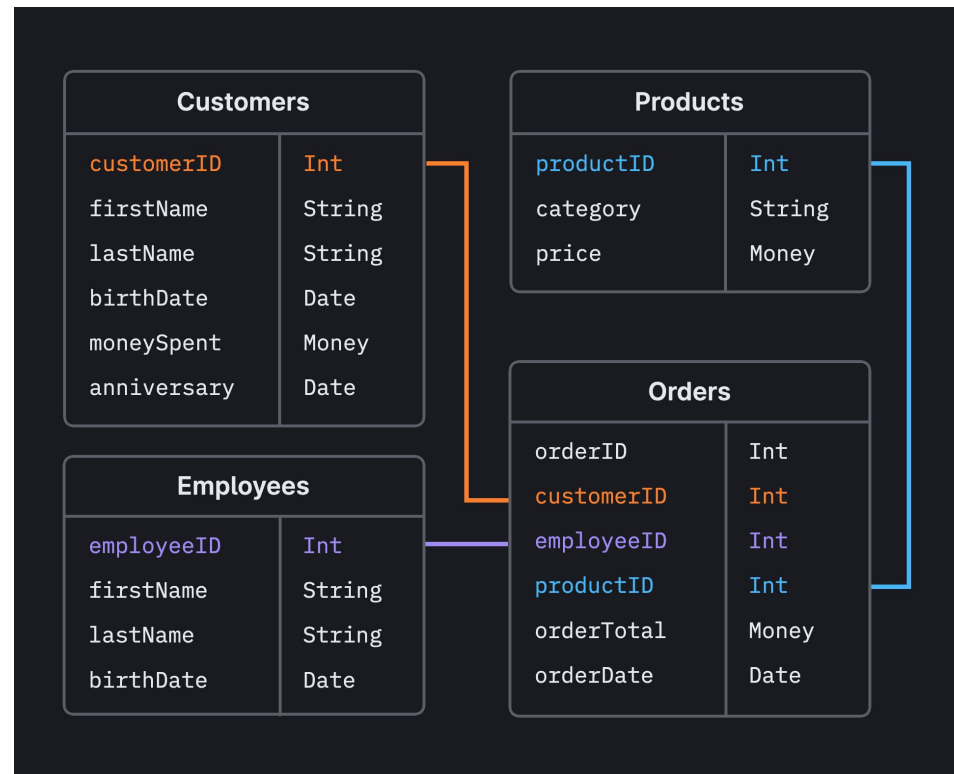


- In modern infrastructure, databases generally fall into two categories: Relational (SQL) and Non-Relational (NoSQL).
- SQL (Relational): Data is structured in strict tables, rows, and columns. Relationships are strictly defined.
- NoSQL (Non-Relational): Data is flexible. Often stored as JSON-like documents, key-value pairs, or wide-columns.
- DevOps Engineers must know how to provision and support both!



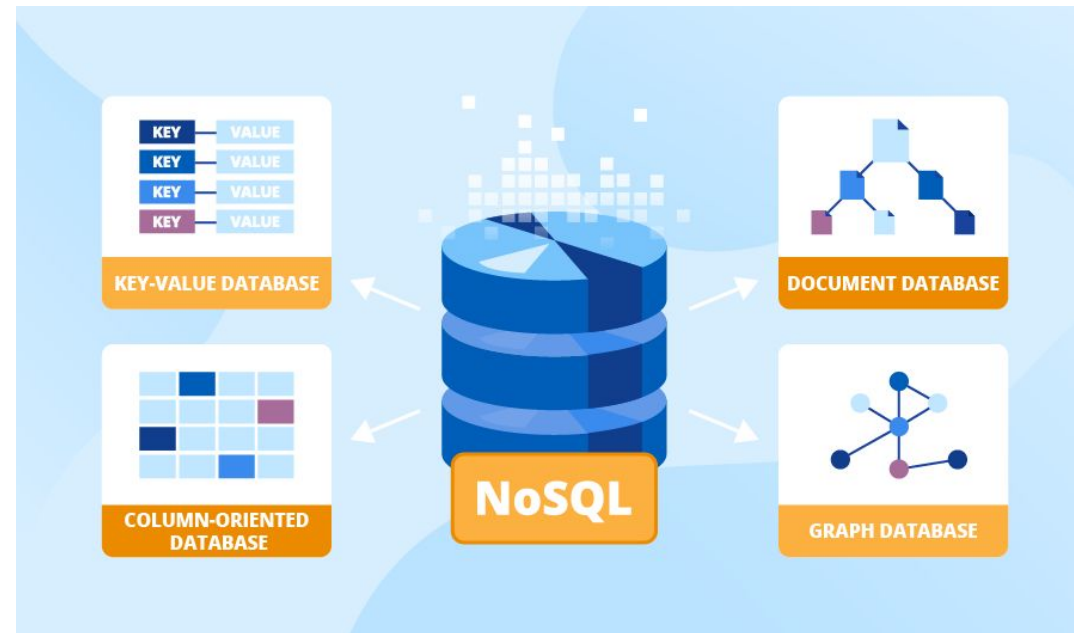


- Examples: PostgreSQL, MySQL, Oracle, Microsoft SQL Server.
- Use Case: Financial systems, e-commerce, complex transactions where data integrity is critical (ACID compliance).
- Structure: Requires a predefined schema (you must define the columns before inserting data).
- Scaling: Traditionally scales vertically (buying a bigger, stronger server).





- Examples: MongoDB (Document), Redis (Key-Value), AWS DynamoDB, Cassandra.
- Use Case: Real-time analytics, caching, gaming leaderboards, rapid prototyping.
- Structure: Schema-less or dynamic. You can insert a JSON document with 3 fields, and another with 10 fields into the same collection.
- Scaling: Designed to scale horizontally (adding more cheap servers to a cluster).





Why SQLite for Today?

- For "Project Genesis," our ultimate goal is AWS RDS (PostgreSQL).
- However, running a full PostgreSQL server on your laptop just to write Python code is overkill.
- SQLite: A C-library that provides a lightweight, disk-based SQL database. It saves everything to a single file (e.g., app.db).
- The DevOps Strategy: We use SQLite for local development and testing, and swap it to PostgreSQL in Production by changing one single line of config.

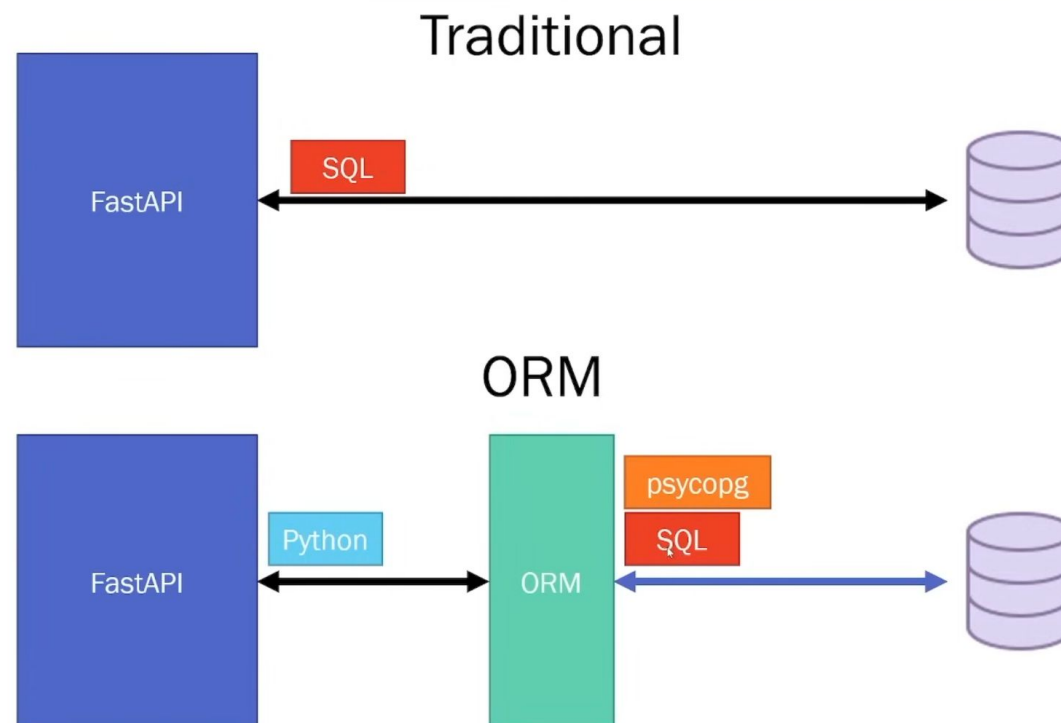


- Historically, developers wrote raw SQL queries inside Python strings to talk to the database.
- Issues: Prone to SQL injection attacks, hard to maintain, and ties your code to a specific database dialect (MySQL syntax vs Postgres syntax).
- It breaks the "Object-Oriented" paradigm of Python.



Enter the ORM (Object-Relational Mapper)

- An ORM is a library that translates Python classes into Database tables.
- Python Objects = Database Rows.
- Python Attributes = Database Columns.
- Allows you to interact with your database using pure Python, without writing raw SQL.





- The most popular, enterprise-grade ORM for Python.
- Database Agnostic: Write your database logic once. SQLAlchemy translates it to SQLite, PostgreSQL, or MySQL under the hood!
- Provides built-in security against SQL injection.
- Integrates beautifully with FastAPI and Flask.



```
1 # Install it via pip
2 pip install sqlalchemy
```

```
from datetime import datetime

from sqlalchemy import (create_engine, desc,
                        Index, Column, DateTime, Integer, String)
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker

Base = declarative_base()

class Student(Base):
    __tablename__ = 'students'

    id = Column(Integer(), primary_key=True)
    name = Column(String())

if __name__ == '__main__':
    engine = create_engine('sqlite:///memory:')
    Base.metadata.create_all(engine)

    # use our engine to configure a 'Session' class
    Session = sessionmaker(bind=engine)
    # use 'Session' class to create 'session' object
    session = Session()
```



- The Engine is the starting point for any SQLAlchemy application.
- It manages the database connection pool and speaks the specific dialect (in today's case, SQLite).
- We define it using a Connection String (URI).

```
>>> from sqlalchemy import create_engine, text
...
... # Make the engine
... engine = create_engine("sqlite+pysqlite:///vball.db", future=True)
...
... # Use a context manager
... with engine.connect() as conn:
...     result = conn.execute(text("SELECT 'hello world'"))
...     print(result.all())
...
... [ ('hello world',) ]
>>>
```



- The Session: Your "workspace" for database operations. You add things here and `commit()` them.
- The Base: All our Python models will inherit from this class. It keeps track of our tables so we can generate the database file later.



```
1  from sqlalchemy.orm import sessionmaker, declarative_base
2
3  # The Workspace
4  SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
5
6  # The Registry
7  Base = declarative_base()
```



- A Model is just a Python Class inheriting from Base.
- We define `__tablename__` to name the SQL table.
- We define Column objects to specify data types (Integer, String, Boolean).
- Every table needs a Primary Key (a unique ID).

```
1  from sqlalchemy import Column, Integer, String
2
3  class User(Base):
4      __tablename__ = "users"
5
6      id = Column(Integer, primary_key=True, index=True)
7      username = Column(String, unique=True, index=True)
8      email = Column(String, unique=True, index=True)
```




Initializing the Tables

- Defining the models doesn't create the `.db` file automatically.
- We must tell SQLAlchemy to bind the models to the engine and actually build the tables.
- Usually, this goes in your main application file before the web server starts.



```
1 # Creates the project_genesis.db file and all tables defined in models
2 Base.metadata.create_all(bind=engine)
```



- Step 1: Create a new Python object from your Model.
- Step 2: Add it to the session (db.add()).
- Step 3: Commit the session (db.commit()).
- Step 4: Refresh the object to get its new auto-generated ID (db.refresh()).



```
1 new_user = User(username="arman", email="arman@devops.com")
2 db.add(new_user)
3 db.commit()
4 db.refresh(new_user)
```



- Use the session to query a specific model.
- `.all()` fetches every record in the table as a Python list.
- `.filter()` applies SQL WHERE clauses to narrow it down.
- `.first()` returns the first match.



```
1  # Get all users (SELECT * FROM users)
2  all_users = db.query(User).all()
3
4  # Get a specific user by ID
5  user = db.query(User).filter(User.id == 1).first()
```



- If 100 users hit your API at once, they shouldn't share the exact same database connection.
- We use FastAPI's Depends to create a fresh Database Session when a route is called, and cleanly close it when the route finishes.



```
1  # Yields a session, then guarantees it closes safely
2  def get_db():
3      db = SessionLocal()
4      try:
5          yield db
6      finally:
7          db.close()
```

- Homework 7: Add the DB's
- Next Lesson: Networking

Questions?

CONTACT US